Snazzy Asserts

Unnumbered PEP draft

PEP	TBD
Author	Pablo Galindo Salgado
Discussions-To	Pending
Status	Draft
Type	Standards Track
Created	18-Oct-2026
Python-Version	3.16

Abstract

This PEP proposes optional compiler support for assertion diagnostics. The compiler emits code that retains subexpression values during execution. On failure, the exception receives those values. The traceback can show them. A new standard library API exposes the same data to test frameworks and other tools.

The feature is off by default. One compiler flag or command-line option enables it. Ordinary assertion removal under `-O` and `-OO` still applies.

Motivation

When an assertion fails, the traceback shows its source but can omit the values needed to explain the failure. A developer with only a CI log or a user report often needs another run with additional diagnostic code.

Consider a check of an invoice total:

```
def check(price, quantity, expected):
    assert price * quantity == expected
```

For `check(18, 3, 60)`, the ordinary traceback omits the operand values. With `python -X assert_details`, it includes values from that execution:

```
AssertionError
  assert (18 * 3) == 60
```

For a function call, the explanation can also identify the return value:

```
def normalize(name):
    return name.strip().lower()

assert normalize(" ALICE ") == "bob"
```

```
AssertionError
  assert 'alice' == 'bob'
    + where 'alice' = normalize(' ALICE ')
    - bob
    + alice
```

Current tools obtain diagnostic values in different ways:

Assertion diagnostics in current tools and this proposal

Approach	Information available	Required work
Plain `assert`	Source line and an optional message.	The author selects values for each message.
`TestCase.assertEqual`	Both arguments and type-specific differences.	Use an assertion method. Intermediate results inside arguments are not available to that method.
pytest rewrite	Values of common subexpressions and custom comparisons.	Rewrite selected modules before compilation.
Compiler capture	Captured values and source locations through a public API.	Enable a compiler flag or command-line option.

The unittest documentation and pytest's assertion documentation describe these approaches. This proposal lets frameworks share the compiler's capture implementation. Diagnostic tools can use the same data outside a test runner.

Barry Warsaw reported this problem in a 2016 issue, after he had to edit installed code to obtain the missing values. A 2025 discussion proposed annotated assertions and considered runtime cost, bytecode size, and output quality.

There is a maintenance cost to a separate rewrite. The Python 3.11.0b3 release report records an expedited release after AST validation changes broke compatibility with pytest. For failure explanations, a framework can use compiler capture instead of replacement AST nodes.

Rationale

The values must come from the original execution. For example:

```
assert read_total() == invoice.total
```

A second call to `read_total()` can consume more input. Another access to `invoice.total` can run a property with side effects. Frame locals do not necessarily contain either result. The compiler must retain them before the comparison consumes them.

PEP 657 provides source positions, but those positions do not retain operand values. PEP 678 provides exception notes and explicitly excludes structured data from their purpose. A library needs the original objects for operations such as an array difference, so formatted text alone is insufficient.

Capture extends object lifetimes and adds work to successful assertions. The assertion-loop measurements below show a 1.86x geometric mean cost. These costs require an explicit opt-in. Successful assertions release the captured values without a callback.

Specification

Enabling assertion diagnostics

Capture is off by default. `python -X assert_details` enables capture and failure explanations. There is no separate disable option.

Libraries select the same feature with `ast.PyCF_ASSERT_DETAILS`:

```
import ast

code = compile(
    "assert price * quantity == expected", "invoice.py", "exec",
    flags=ast.PyCF_ASSERT_DETAILS, dont_inherit=True,
)
```

The flag applies to source or AST input and to nested code in that compilation. Imported modules and later `compile()` calls do not inherit it. AST-only compilation returns an ordinary AST without capture state. The C API accepts `PyCF_ASSERT_DETAILS` in `PyCompilerFlags.cf_flags`. The command-line option applies to source compilation throughout the process, also for calls with `dont_inherit=True`. It cannot change code objects that already exist, frozen modules, or bytecode-only imports.

`-O` and `-OO` still remove assertions with `-X assert_details`. The `optimize` argument to `compile()` follows the ordinary assert statement rules. Import caches must distinguish code with capture from code without it.

Capture semantics

The API represents an assertion as a graph of expression nodes. Each node describes a subexpression, links to its children, and exposes its captured result when available.

Capture preserves evaluation order and short circuit behavior. It does not repeat calls, comparisons, or truth conversion, and does not call `repr()`. For `assert left() and right()`, if `left()` is false, the graph records its result and marks `right()` as unavailable. A formatter must not evaluate the skipped call.

Captured values are references to the original objects. Later mutation can change what a report shows. Capture also extends object lifetimes and can change the assertion's result. After failure, the exception and any retained details objects can keep the captured objects alive.

Suppose `make()` returns a weak-referenceable object with no other references. CPython normally destroys the temporary before the last call in `assert weakref.ref(make())() is None`, so that assertion passes. A capture slot keeps it alive, so the assertion fails.

A comparison can return an object other than `bool`. The API exposes the raw result as `value` and the truth result used by execution as `truth`. For `a < b < c`, each link has a separate record and `b` evaluates once. An available value of `None` differs from an unavailable value.

The graph describes the assertion test. It does not trace called functions or individual iterations within comprehensions. Constant folding can remove nodes before capture. Expression text is normalized and need not preserve spaces or quotes. `await` and `yield` remain permitted, with captures in the suspended frame until execution resumes or cleanup releases them.

The optional assertion message evaluates once, after the condition fails. If the condition or message raises, that exception propagates without a failure callback. An explicit `raise AssertionError(...)` creates no capture data.

The assertionlib module

The proposed module name is `assertionlib`, subject to the name conflict under Open Issues. Its operations are:

```
get_details(error) -> AssertionDetails | None
clear_details(error) -> None
format_details(details, *, compare=None) -> list[str]
handlers(*, on_failure=None)  # context manager
```

`get_details()` builds and caches the details object on its first call. Later calls return the same object. It returns `None` if capture data is absent. `clear_details()` detaches that data and the cached graph. It does not clear the traceback or other references to the graph. Graph attributes are immutable.

An `AssertionDetails` object exposes `expression`, `location`, `root`, `nodes`, and `comparisons`. Each expression node has an ID, text, kind, location, children, and capture data. Check its `evaluated` attribute before access to `value`, which otherwise raises `ValueError`. `truth` is `None` when unavailable and never calls `bool()` to reconstruct a result.

Node IDs identify expression occurrences within one graph, independently of frame-slot numbers. A comparison record exposes `operator`, `left`, `right`, `expression`, `text`, and the same capture attributes. A source location gives the filename, start and end lines, and zero-based UTF-8 column offsets. End offsets are exclusive. The technical notes give the full attribute and error contracts.

`format_details()` returns lines with terminal newlines and performs no output. It can call user `__repr__` methods.

The optional `compare(op, left, right)` callback receives captured operands for each evaluated comparison. Return `None` for the default explanation, lines for a custom explanation, or an empty iterable to suppress the default difference. The first custom line replaces the comparison summary. Later lines supply detail. Custom lines have no terminal newlines. Formatter errors propagate to the caller.

For the earlier compilation, a failure callback can inspect the values:

```
import assertionlib

def report_failure(error, details):
    comparison = details.comparisons[0]
    print(comparison.left.value, comparison.operator,
          comparison.right.value)

with assertionlib.handlers(on_failure=report_failure):
    exec(code, {"price": 18, "quantity": 3, "expected": 60})
```

This prints `54 == 60` before the `AssertionError` propagates. A callback can change `error.args`, but its return value cannot suppress the failure. If it raises, its exception retains the assertion error as `__context__`. This rule also applies to failures in diagnostic support. During the callback, `sys.exception()` is the assertion error, which has no traceback yet.

`handlers()` registers a synchronous failure callback in the current execution context. It does not enable capture. A nested `handlers()` block replaces the handler. An omitted argument means no handler inside that block. Exit restores the previous handler.

Context copies and new asyncio tasks inherit the handler, under the normal contextvars rules. Recursive assertions inside a callback do not dispatch another callback.

Traceback output and exception serialization

Capture alone leaves `AssertionError.args` and `str(error)` unchanged. The standard traceback formatter adds the explanation after the exception message and before its notes. This also applies within exception groups. Exact whitespace and difference layout are not API guarantees. `TracebackException` retains formatted lines rather than captured objects.

The automatic traceback path catches `Exception` from the explanation formatter. It keeps the ordinary report and adds an unavailable-data line.

The proposed exception pickle protocol omits capture data and the cached graph, while it preserves ordinary arguments and user attributes.

CPython implementation

The parser produces an ordinary `Assert` node. Capture takes place during bytecode generation, after the AST and symbol-table passes. `ast.unparse()` still shows the original assertion. The Python below is pseudocode. Its names represent hidden slots and internal operations.

Compiler transformation

The assertion in `check()` starts as:

```
assert price * quantity == expected
```

The compiler retains each result before the next operation consumes it:

```
try:
    price_value = price                     # slot 3
    quantity_value = quantity               # slot 4
    product = price_value * quantity_value  # slot 5
    expected_value = expected               # slot 6
    comparison = product == expected_value  # slot 7
    passed = bool(comparison)               # slot 8
    if not passed:
        error = AssertionError()
        attach_and_dispatch(error, capture_plan)
        raise error
finally:
    clear_capture_slots()
```

`bool()` denotes the interpreter's truth conversion, without a name lookup. Cleanup releases every capture on success or an exception. Empty slots are valid when evaluation stops early.

The temporaries occupy hidden slots in the frame's `localsplus` array, after the three arguments. A constant plan in `co_consts` maps each slot to its expression node, text, and location. Slot 5 maps to `price * quantity`. The compiler obtains normalized text from its AST unparser. Internal slot names appear in `co_varnames`, but remain hidden from `locals()`.

Values must remain available after later expressions execute. In `assert x == (x := 2)`, the local `x` no longer holds the left operand when the assertion fails. A saved reference preserves that operand.

For short circuit expressions, the compiler places captures inside each branch. In `assert left() and right()`, a false result from `left()` skips both `right()` and its captures. Comparison chains retain a separate result for each evaluated link.

Failure callbacks

On failure, `attach_and_dispatch()` attaches the plan and captured references to the exception before cleanup. With the earlier handler installed, its dispatcher makes this ordinary Python call:

```
details = assertionlib.get_details(error)
report_failure(error, details)  # prints: 54 == 60
```

`get_details()` builds the graph from the plan and saved values. The comparison's `left.value` comes from slot 5 (`54`), and its `right.value` comes from slot 6 (`60`).

`handlers()` stores the callback in a `ContextVar`. The prototype finds an already imported `assertionlib` module with PyImport_GetModule(), then calls its dispatcher. The dispatcher checks the current handler and prevents recursive callbacks. The plan contains no functions or imports.

Without a handler, failures still retain capture data. Successful assertions create no diagnostic containers and perform no callback lookup.

Module lookup is a prototype detail. Removal of `assertionlib` from `sys.modules` disables dispatch. The traceback formatter can import support when it first needs an explanation.

Bytecode and runtime support

Capture and cleanup use instructions that CPython already has:

- `STORE_FAST slot; LOAD_FAST slot` retains the value in a hidden local and returns a reference to the operand stack. The optimizer can combine these operations into `STORE_FAST_LOAD_FAST`.
- `LOAD_FAST_AND_CLEAR slot; POP_TOP` clears the slot and releases its reference. It also accepts an empty slot after a skipped branch.

The pseudocode's `attach_and_dispatch()` is the C helper `INTRINSIC_ASSERT_DETAILS`. CPython's CALL_INTRINSIC_2 instruction calls it with the exception and plan. It returns the exception for `RAISE_VARARGS`. No new call instruction is necessary.

These sequences use CPython's ownership rules and optimizer. Other Python implementations can use a different internal representation. The technical notes retain the bytecode sequence and internal layout.

Performance

Capture adds work even when assertions pass. Across eleven assertion-loop benchmarks, its geometric mean execution time is 1.86x that of ordinary assertions, compared with 1.65x for pytest's rewrite.

The table compares compilation of 612 SymPy 1.14.0 test files in the same interpreter. Times are medians. Serialized size includes diagnostic metadata.

Compilation of SymPy test sources

Mode	Compile, s	Bytecode, MiB	Serialized, MiB
Ordinary asserts	1.26	12.08	20.57
pytest rewrite	48.20	89.33	131.90
Compiler capture	9.64	25.90	95.48

For this corpus, capture uses 80% less compile time than pytest. It still takes 7.7x the compile time of ordinary assertions and produces 4.64x the serialized size.

Earlier runs of 205 selected SymPy tests on a shared host did not establish a stable runtime advantage over pytest. The research notes contain the full results, measurement method, and reproduction scripts.

Backwards Compatibility

With capture disabled, assertions retain their ordinary compile mode. The prototype's cost in that mode, relative to unmodified CPython, remains unmeasured.

Opt-in capture changes object lifetimes and can fail to allocate memory. Callbacks and formatters can execute user code. Extra traceback lines affect exact output checks such as `doctest`. The prototype cleanup handler also adds `EXCEPTION_HANDLED` and `RERAISE` monitoring events after a failed assertion. It consumes one compiler block and can reach the static block limit sooner.

The new intrinsic and capture layout require a bytecode magic-number change. Opcode numbers stay unchanged. Private capture layouts remain implementation details.

Security Implications

Captured values can contain credentials or other private data. Reports need the same redaction policy as other application diagnostics. A representation length limit does not limit execution time in `__repr__`. Assertions remain unsuitable for checks that must execute under every optimization mode.

Rejected Ideas

A new assertion function receives arguments after their evaluation and cannot inspect their intermediate results. The assertion syntax already identifies the expression that needs capture.

PEP 669 monitoring events offer another callback mechanism, but an event cannot recover operands after their consumption. Compiler capture is still necessary. The proposed callbacks use context-local scopes, so separate tasks can install different handlers.

How to Teach This

Start with the two traceback examples and `python -X assert_details`. For library authors, show the compile flag followed by the callback example. Explain that a handler does not enable capture and that values are live references.

Reference Implementation

Prototype implementation.

Open Issues

- Select a module name. The AssertionLib package already uses `assertionlib`.
- Define public suppression of the automatic traceback explanation when a framework already places one in `error.args`. Include rules for `doctest` output and framework reports.
- Specify minimum graph coverage across compiler optimizations.
- Reduce compile time and serialized size for code with many assertions.
- Measure cold imports, peak memory, end-to-end failures, and representative applications on a controlled host.

Copyright

This document is placed in the public domain or under the CC0-1.0-Universal license, whichever is more permissive.